Computational Gastronomy — Assignment 2

Question 3: Size- and Ingredient Frequency-Controlled Random Control

Monsoon 2026

Problem Statement

Create size- and ingredient frequency-controlled random control of the above 10,000 recipes assuming them to be a cuisine. Create 10 sets of random cuisines.

- **(a)** Plot recipe size distribution of the original cuisine and & random recipes. [5]
- **(b)** Plot frequency-rank distribution of the original cuisine & the random recipes. [5]

In [1]:
```
import sys
import os
import time
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Styling
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.styl
plt.rcParams['font.sans-serif'] = ['DejaVu Sans', 'Arial', 'Helvetica']
plt.rcParams['axes.edgecolor'] = '#cccccc'
plt.rcParams['axes.linewidth'] = 0.8

DATA_PATH = Path('../Group_23_Assignment1/Q1b_ingredients_ner.csv')
RANDOM_SEED = 20260929
np.random.seed(RANDOM_SEED)
```

In [2]:
```
# Load data
ner_df = pd.read_csv(DATA_PATH, keep_default_na=False)
valid_ner = ner_df[ner_df['Ingredient Name'].str.strip() != '']

# Vocabulary mapping
vocab = sorted(list(set(valid_ner['Ingredient Name'])))
V = len(vocab)
ing2idx = {name: i for i, name in enumerate(vocab)}

# Unique recipes mapping
recipes_dict = valid_ner.groupby('Recipe ID')['Ingredient Name'].apply(lambda
recipe_ids = sorted(list(recipes_dict.keys()))
N = len(recipe_ids)

# Encode recipes as list of sets of ingredient indices
original_recipes_indices = [set(ing2idx[ing] for ing in recipes_dict[rid]) for

                            es
```

Loading [MathJax]/extensions/Safe.js

```
orig_recipe_sizes = np.array([len(r) for r in original_recipes_indices])
orig_ingredient_freqs = np.zeros(V, dtype=int)
for r in original_recipes_indices:
    for ing_idx in r:
        orig_ingredient_freqs[ing_idx] += 1

print(f"Recipes N: {N:,}, Vocabulary V: {V:,}")
print(f"Total recipe-ingredient edges: {orig_recipe_sizes.sum():,}")
print(f"Mean recipe size: {orig_recipe_sizes.mean():.4f}")
print(f"Top 5 most frequent ingredients:")
top5_indices = np.argsort(orig_ingredient_freqs)[::-1][:5]
for idx in top5_indices:
    print(f"  • {vocab[idx]}: {orig_ingredient_freqs[idx]:,} recipes")
```

```
Recipes N: 10,000, Vocabulary V: 2,425
Total recipe-ingredient edges: 97,190
Mean recipe size: 9.7190
Top 5 most frequent ingredients:
  • garlic clove: 3,047 recipes
  • olive oil: 2,294 recipes
  • butter: 2,011 recipes
  • onion: 1,729 recipes
  • plain flour: 1,677 recipes
```

Null Model Methodology: Curveball Bipartite Graph Randomization

A cuisine can be formally represented as a **bipartite graph** $G = (R, I, E)$, where:

- R is the set of $N = 10,000$ recipes.
- I is the set of $V = 2,425$ unique ingredients.
- An edge $(r, i) \in E$ indicates that recipe r contains ingredient i.

The task requires creating a random null model where **both** of the following margins are strictly controlled:

1. **Recipe size distribution (Row sums):** Every recipe retains its exact number of ingredients.
2. **Ingredient frequency distribution (Column sums):** Every ingredient occurs with its exact original frequency across the entire cuisine.

The Curveball Algorithm (Strona et al., 2014)

To generate an unbiased, uniform sample from the space of all binary matrices with fixed row and column sums, we implement the **Curveball Algorithm**:

- In each step, pick two recipes u and v at random.
- Identify their common ingredients $C = r_u \cap r_v$ and differences $D_u = r_u \setminus C$, $D_v = r_v \setminus C$.
- Combine the non-shared ingredients $D = D_u \cup D_v$ and shuffle them uniformly at

Loading [MathJax]/extensions/Safe.js

- Reassign $|D_u|$ items to u and $|D_v|$ items to v.
- This operation **exactly preserves** the degree of each recipe ($|r_u|$ and $|r_v|$) and the total frequency of each ingredient in D, while thoroughly rewiring the bipartite connections without introducing self-loops or duplicate edges.

In [3]:
```
def run_curveball(initial_recipes, num_passes=15, rng=None):
    """
    Efficient implementation of the Curveball algorithm.
    Preserves exact recipe sizes and ingredient frequencies.
    """
    if rng is None:
        rng = np.random.default_rng()

    recipes = [set(r) for r in initial_recipes]
    n = len(recipes)

    for step in range(num_passes):
        perm = rng.permutation(n)
        for i in range(0, n - 1, 2):
            u, v = perm[i], perm[i+1]
            A, B = recipes[u], recipes[v]
            common = A & B
            diff_A = list(A - common)
            diff_B = list(B - common)
            total_diff = diff_A + diff_B
            if total_diff:
                rng.shuffle(total_diff)
                nA = len(diff_A)
                recipes[u] = common | set(total_diff[:nA])
                recipes[v] = common | set(total_diff[nA:])
    return recipes

# Generate 10 sets of randomized cuisines
num_sets = 10
random_cuisines = []

print("Generating 10 randomized cuisines using Curveball algorithm...")
t0 = time.time()
for s in range(num_sets):
    rng_s = np.random.default_rng(seed=RANDOM_SEED + s * 1000)
    rand_recipes = run_curveball(original_recipes_indices, num_passes=15, rng=
    random_cuisines.append(rand_recipes)

    # Verify conservation of degree sequences
    rand_sizes = np.array([len(r) for r in rand_recipes])
    rand_freqs = np.zeros(V, dtype=int)
    for r in rand_recipes:
        for ing_idx in r:
            rand_freqs[ing_idx] += 1

    assert np.array_equal(rand_sizes, orig_recipe_sizes), f"Set {s+1} recipe s
    assert np.array_equal(rand_freqs, orig_ingredient_freqs), f"Set {s+1} ingr
    print(f"  • Random Cuisine Set {s+1:2d} generated and verified (max size d
t1 = time.time()
print(f"All 10 sets generated in {t1 - t0:.2f}s")
```

Loading [MathJax]/extensions/Safe.js

```
Generating 10 randomized cuisines using Curveball algorithm...
  • Random Cuisine Set  1 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  2 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  3 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  4 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  5 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  6 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  7 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  8 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set  9 generated and verified (max size diff: 0, max freq d
iff: 0)
  • Random Cuisine Set 10 generated and verified (max size diff: 0, max freq d
iff: 0)
All 10 sets generated in 2.62s
```

Q3(a) — Recipe Size Distribution of Original Cuisine & Random Recipes

We plot the recipe size distribution comparing the original cuisine against each of the 10 random cuisines. Because the Curveball degree-preserving null model fixes row sums exactly, all 10 sets overlap precisely with the original distribution.

In [4]:

```
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

max_size = orig_recipe_sizes.max()
bins = np.arange(0.5, max_size + 1.5, 1)
bin_centers = np.arange(1, max_size + 1)

palette = plt.cm.tab10(np.linspace(0, 1, 10))

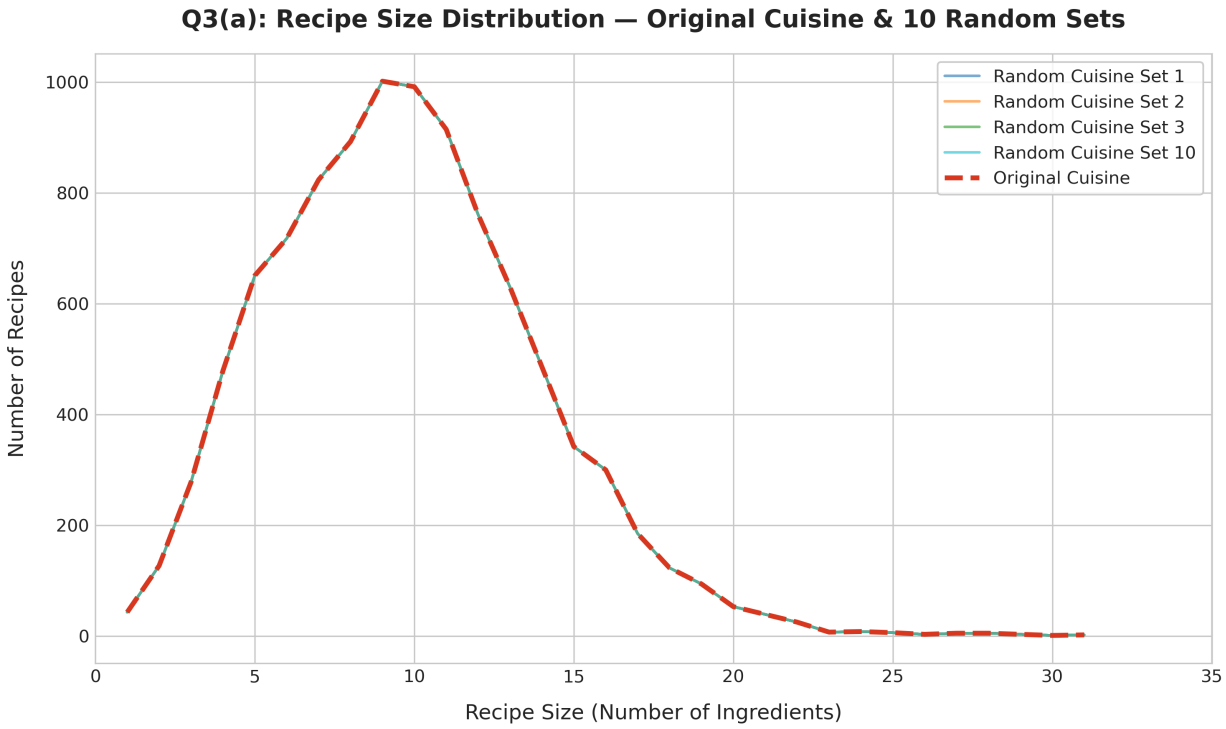
# Plot each of the 10 random sets
for s in range(num_sets):
    rand_sizes = np.array([len(r) for r in random_cuisines[s]])
    counts, _ = np.histogram(rand_sizes, bins=bins)
    ax.plot(bin_centers, counts, color=palette[s], alpha=0.6, linewidth=1.5,
            label=f'Random Cuisine Set {s+1}' if s < 3 or s == 9 else "")

# Plot original cuisine
orig_counts, _ = np.histogram(orig_recipe_sizes, bins=bins)
ax.plot(bin_centers, orig_counts, color='#d9381e', linewidth=2.8, linestyle='-
        label='Original Cuisine', zorder=10)

ax.set_title('Q3(a): Recipe Size Distribution — Original Cuisine & 10 Random Se
ax.set_xlabel('Recipe Size (Number of Ingredients)', fontsize=12, labelpad=10)
ax.set_ylabel('Number of Recipes', fontsize=12, labelpad=10)
ax.set_xlim(0, 35)
on=True, facecolor='white', framealpha=0.9, fontsize=10)
plt.tight_layout()
```

Loading [MathJax]/extensions/Safe.js

```
plt.savefig('Q3a_recipe_size_distribution.png', dpi=300)
plt.show()
```



Q3(b) — Frequency-Rank Distribution of Original Cuisine & Random Recipes

We compute the frequency of every ingredient across all recipes, sort them descending to determine their rank $r = 1, 2, \dots, |V|$, normalize by the total number of ingredient mentions: $$f(r) = \frac{\text{Frequency}(r)}{\sum_{i} \text{Frequency}(i)}$$ and plot the frequency-rank distribution on a **log-log scale** (Zipfian plot) for both the original cuisine and the 10 random cuisines.

In [5]:
```
# Compute frequency-rank for original cuisine
orig_sorted_freqs = np.sort(orig_ingredient_freqs)[::-1]
total_occurrences = orig_sorted_freqs.sum()
orig_norm_freqs = orig_sorted_freqs / total_occurrences
ranks = np.arange(1, len(orig_sorted_freqs) + 1)

# Plot Frequency-Rank log-log distribution
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

for s in range(num_sets):
    rand_freqs = np.zeros(V, dtype=int)
    for r in random_cuisines[s]:
        for ing_idx in r:
            rand_freqs[ing_idx] += 1
    rand_sorted = np.sort(rand_freqs)[::-1]
    rand_norm = rand_sorted / rand_sorted.sum()
    ax.plot(ranks, rand_norm, color=palette[s], alpha=0.5, linewidth=1.2,
            label=f'Random Cuisine Set {s+1}' if s < 3 or s == 9 else "")
```

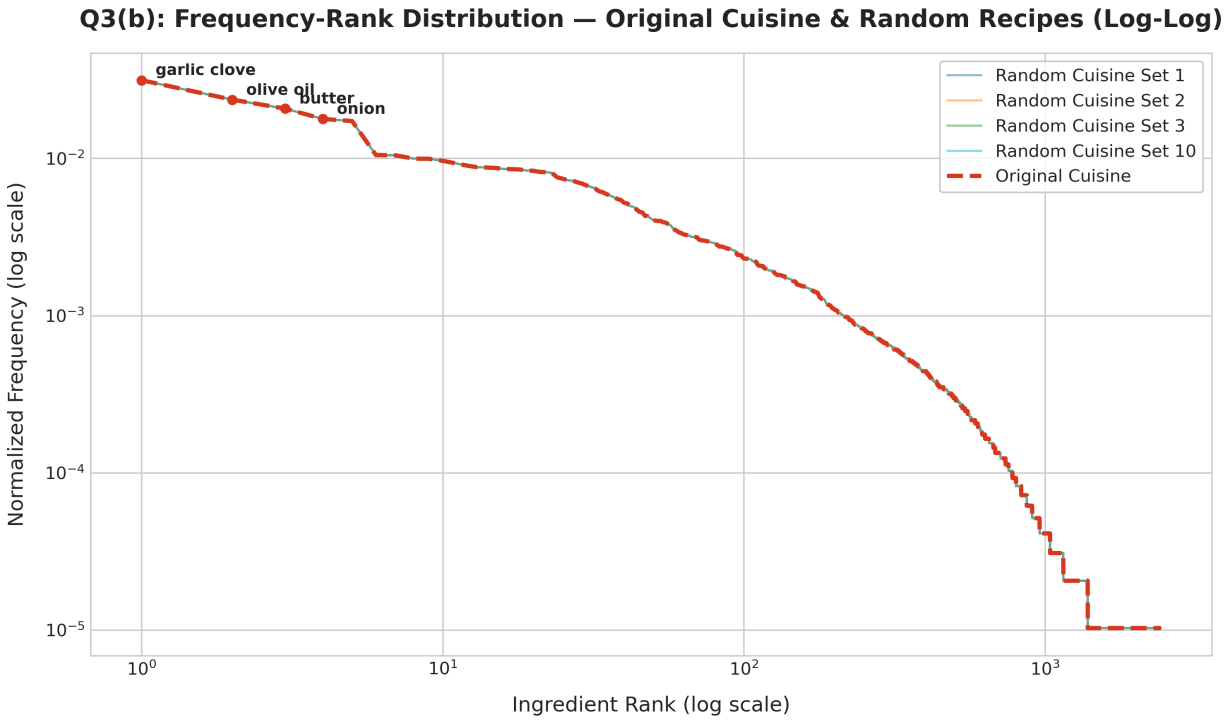
Loading [MathJax]/extensions/Safe.js

```
ax.plot(ranks, orig_norm_freqs, color='#d9381e', linewidth=2.5, linestyle='--'
        label='Original Cuisine', zorder=10)

# Annotate prominent staple ingredients
staples = [
    (1, vocab[top5_indices[0]], orig_norm_freqs[0]),
    (2, vocab[top5_indices[1]], orig_norm_freqs[1]),
    (3, vocab[top5_indices[2]], orig_norm_freqs[2]),
    (4, vocab[top5_indices[3]], orig_norm_freqs[3]),
]
for rank_pos, name, val in staples:
    ax.scatter(rank_pos, val, color='#d9381e', s=25, zorder=12)
    ax.annotate(f" {name}", (rank_pos, val), fontsize=9, fontweight='semibold'
                xytext=(5, 3), textcoords='offset points')

ax.set_xscale('log')
ax.set_yscale('log')
ax.set_title('Q3(b): Frequency-Rank Distribution — Original Cuisine & Random Re
ax.set_xlabel('Ingredient Rank (log scale)', fontsize=12, labelpad=10)
ax.set_ylabel('Normalized Frequency (log scale)', fontsize=12, labelpad=10)
ax.legend(frameon=True, facecolor='white', framealpha=0.9, fontsize=10)
plt.tight_layout()

plt.savefig('Q3b_frequency_rank_distribution.png', dpi=300)
plt.show()
```



Question 3 Deliverables Summary

Loading [MathJax]/extensions/Safe.js

Deliverable File	Description	Question
`Q3a_recipe_size_distribution.png`	Recipe size distribution of original cuisine vs 10 size- and frequency-controlled random cuisines	Q3(a)
`Q3b_frequency_rank_distribution.png`	Log-log Zipfian frequency-rank distribution of original cuisine vs 10 random cuisines	Q3(b)

Loading [MathJax]/extensions/Safe.js